Java LinkedList

Study Notes: Overview, Internals, Use Cases & Time Complexity

1. Overview

- A class in the `java.util` package.
- **Not thread-safe**.
- Implements both the `List` and `Deque` interfaces, so it supports list-like and queue/deque-like operations.
- Unlike `ArrayList`, **it does not use an array**. It uses **nodes**, where each node contains:
  - The data (value)
  - A reference to the next node
  - A reference to the previous node (Java uses a **doubly linked list**)

2. How It Works Internally

- Each element is a **node**, and Java maintains:
  - A reference to the **head** (first element)
  - A reference to the **tail** (last element)
- When you add/remove elements at the start or end:
  - It only updates a couple of pointers, so it is very fast: **O(1)**

3. When to Use LinkedList

- **Queue or Stack behavior**
  - Use `addFirst()`, `addLast()`, `removeFirst()`, `removeLast()`
- **Frequent insertions/removals** at the beginning (or middle, when you already hold the position)
- **Memory-sensitive scenarios** where resizing arrays would be too costly
- **Implementing algorithms like:**
  - BFS with queues
  - Undo operations
  - Browser history

4. Time Complexity

Operation	Time Complexity	Explanation
`addFirst()`	O(1)	Adds node at head
`addLast()`	O(1)	Adds node at tail
`add(index, element)`	O(n)	Walks to index

Operation	Time Complexity	Explanation
`get(index)`	O(n)	Traverses from head/tail
`set(index, element)`	O(n)	Same as get
`removeFirst()`	O(1)	Removes from head
`removeLast()`	O(1)	Removes from tail
`remove(index)`	O(n)	Walks to index
`contains(element)`	O(n)	Scans all nodes

Quick Summary

- **O(1):** operations at the ends (head/tail).
- **O(n):** anything that needs to find a position by index or search for a value.